

IT6500: Cloud Computing

for postgraduate programme

Lecture 4

Distributed Computing and Storage

Dr. Nguyen Binh Minh

CLOUD COMPUTING

Cloud Computing: Old Idea Whose Time Has Come

In the early days, the vision was that computing would be a central resource

- John McCarthy (Stanford professor and inventor of Lisp; proposed the 'service bureau' model in 1961)
- J.C.R. Licklider (contributed key ideas to ARPANET; published a memo on the "Intergalactic Computer Network" in 1963)
- Douglas Parkhill (published a book on "The Challenge of the Computer Utility" in 1966)

Famous quote from IBM President Thomas Watson, 1943: “I think there is a world market for maybe five computers”

Earlier attempts for shared infrastructure: mainframes, “the grid”

- Key focus: sharing of *expensive* computers that weren't being maximally used

History: Becoming a cloud provider

Technology	Cost in medium DC (~1,000 servers)	Cost in large DC (~50,000 servers)	Ratio
Network	\$95 per Mbit/sec/month	\$13 per Mbit/sec/month	7.1
Storage	\$2.20 per GByte/month	\$0.40 per GByte/month	5.7
Administration	~140 servers/admin	>1,000 servers/admin	7.1

Source: James Hamilton's Keynote, LADIS 2008

Early 2000s: Phenomenal growth of web services

- Many large Internet companies deploy huge data centers, develop scalable software infrastructure to run them
- Due to economies of scale, these companies were now able to run computation very cheaply
- What else can we do with this?

History: Incentives

Idea: Use your existing data center to provide cloud services

Why is this a good idea?

Make a lot of money

- Price advantage of 3x-7x → Can offer services much cheaper than medium-size company and still make profit

Leverage existing investment

- New revenue stream at low incremental cost (example: many Amazon AWS technologies were initially developed for Amazon's internal operations)

Defend a franchise

- Example: Microsoft enterprise + development apps → Microsoft Azure

History: The pioneers

Jul 2002: Amazon Web Services launched

- Third-party sites can search and display products from Amazon's web site, add items to Amazon shopping carts
- Available through XML and SOAP

Mar 2006: Amazon S3 launched

- Innovative 'pay-per-use' pricing model, which is now the standard in cloud computing
- Cheaper than many small/medium storage solutions: \$0.15/GB/month of storage, \$0.20/GB/month for traffic
- Amazon no longer a pure retailer, entering technology space

Aug 2006: EC2 launched

- Core computing infrastructure becomes available

History: Widespread adoption

Apr 2008: Google App Engine launched

- Same building blocks Google uses for its own applications: Bigtable and GFS for storage, automatic scaling and load balancing, ...

Nov 2009: Windows Azure Beta launched

- Becomes generally available in 21 countries in Feb 2010
- Microsoft's online services are gradually transitioning to Azure

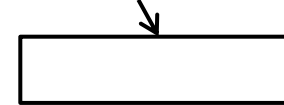
Dec 2013: Google Compute Engine launched

- Provides lower level support vs. App Engine, gives full set of services
- Dramatically lower prices, quickly matched by AWS and Azure

**ONE SET OF CLOUD SERVICES:
AMAZON WEB SERVICES**

Why Amazon AWS and not ?

Insert your favorite
cloud here



Amazon is only one of several cloud providers

- Others include Microsoft Azure, Google Cloud Engine / App Engine, ...

There is no common standard (yet)

- Initially, MS and Google supported PaaS (.NET and Java, resp.)
- Gradually each has grown to support both IaaS and PaaS
- AWS is PaaS/IaaS with a broad menu of choices

So we had to pick one specific provider

- Amazon AWS is going to be used for the rest of this class
 - Amazon's only involvement is providing free AWS credits/storage
- Everything we do on AWS has an equivalent on Azure and GCE/GAE

What is Amazon AWS?

Amazon Web Services (AWS) provides a number of different services, including:

- Amazon Elastic Compute Cloud (EC2)
Virtual machines for running custom software
- Amazon Simple Storage Service (S3)
Simple key-value store, accessible as a web service
- Amazon DynamoDB
Distributed “NoSQL” database, one of several in AWS
- Amazon Elastic MapReduce
Scalable MapReduce computation
- Amazon Mechanical Turk (MTurk)
A 'marketplace for work'
- Amazon CloudFront
Content delivery network
- ...

AWS credentials

Command-line tools
SOAP APIs

AWS web site and
management console

The screenshot shows the AWS sign-in page. It has a header with the Amazon logo and 'Sign In or Create an AWS Account'. Below this, it says 'You may sign in using your existing Amazon.com account or you can create a new account by selecting "I am a new user."'.

My e-mail address is:

☐ I am a new user.

☒ I am a returning user and my password is:

[Sign in using our secure server](#)

[Forgot your password?](#)

[Has your e-mail address changed?](#)

Learn more about [AWS Identity and Access Management](#) and [AWS Multi-Factor Authentication](#) features that provide additional security for your AWS Account.

Sign-in credentials

Access Credentials

There are three types of access credentials used to authenticate your requests to AWS services: (a) access keys, (b) X.509 certificates, and (c) key pairs. Each access credential type is explained below.

The screenshot shows the 'X.509 Certificates' tab in the AWS console. It has a header with 'Access Keys', 'X.509 Certificates', and 'Key Pairs'. Below this, it says 'Use X.509 certificates to make secure SOAP protocol requests to AWS service APIs.'.

Exceptions: Amazon S3 and Amazon Mechanical Turk instead require your [Access Keys](#) for SOAP requests.

Created	X.509 Certificate	Status
	cert-[REDACTED].pem Download	Active (Make Inactive)

[Create a new Certificate](#) | [Upload Your Own Certificate](#)

For your protection, AWS doesn't ask for your private key or retain it on file. You should also never share your private key with anyone. In addition, industry best practice recommends frequent certificate rotation.

[Learn more about X.509 Certificates](#)

X.509 certificates

The screenshot shows the 'Key Pairs' page in the AWS Management Console. It has a header with 'AWS Management Console' and 'Amazon Elastic Compute Cloud (EC2)'. Below this, it says 'Key Pairs'.

[Create Key Pair](#) [Import Key Pair](#) [Delete](#)

Viewing: All Key Pairs Search

Key Pair Name	Fingerprint
ahae-ec2	db:41:f8:06:42:d7:71:4a:62:d2:bb:91:e4:6d:94:cc:ff:66:ca:f9

EC2 key pairs

Access Credentials

There are three types of access credentials used to authenticate your requests to AWS services: (a) access keys, (b) X.509 certificates, and (c) key pairs. Each access credential type is explained below.

The screenshot shows the 'Access Keys' tab in the AWS console. It has a header with 'Access Keys', 'X.509 Certificates', and 'Key Pairs'. Below this, it says 'Use access keys to make secure REST or Query protocol requests to any AWS service API. We create one for you when your account is created — see your access key below.'.

[Your Access Keys](#)

Created	Access Key ID	Secret Access Key	Status
	[REDACTED]	Show	Active (Make Inactive)

[Create a new Access Key](#)

For your protection, you should never share your secret access keys with anyone. In addition, industry best practice recommends frequent key rotation.

[Learn more about Access Keys](#)

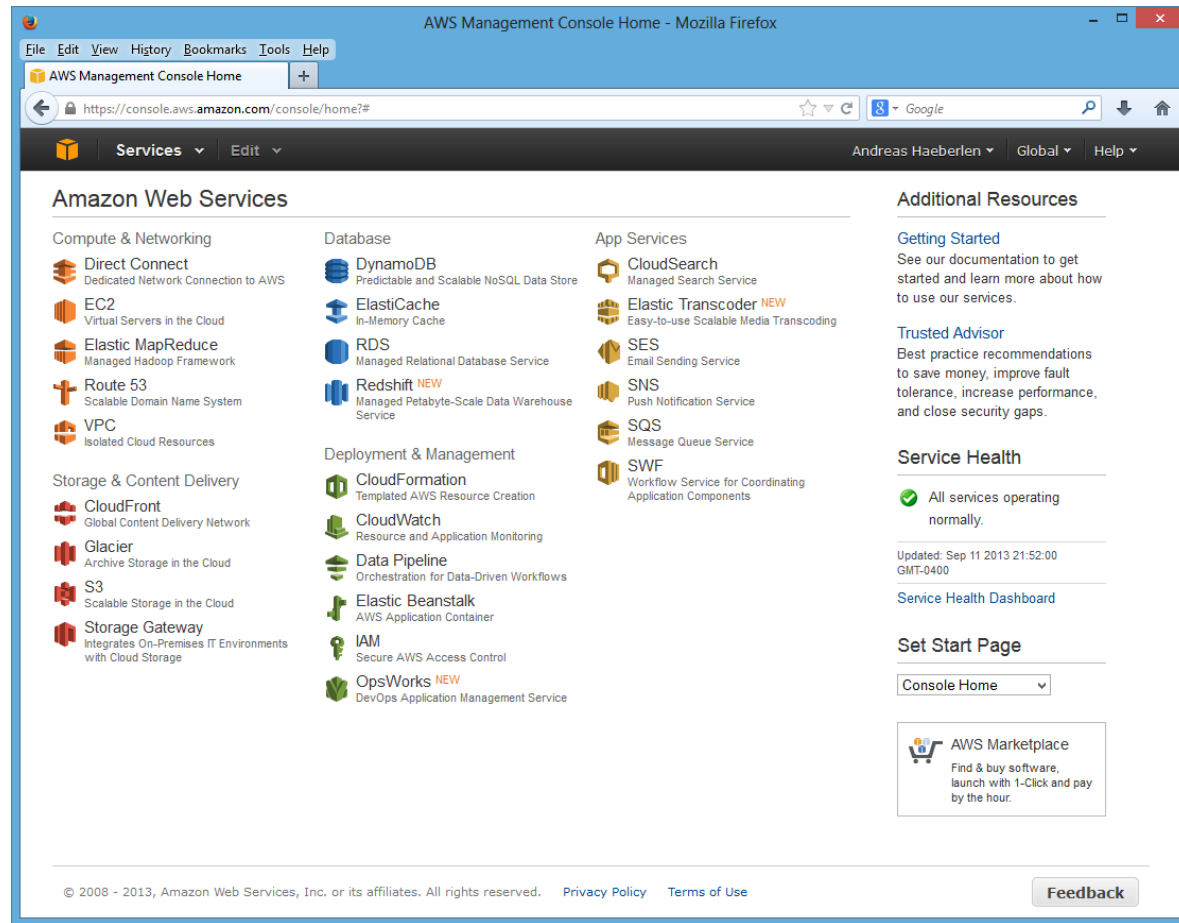
Access keys

REST APIs

Connecting to an
instance (e.g., via ssh)

Why so many different types of credentials?

The AWS management console



Used to control many AWS services:

- For example, start/stop EC2 instances, create S3 buckets...

Requests and Data: REST and SOAP

How do your programs access AWS?

- Via the REST or SOAP protocols
- Example: Launch an EC2 instance, store a value in S3, ...

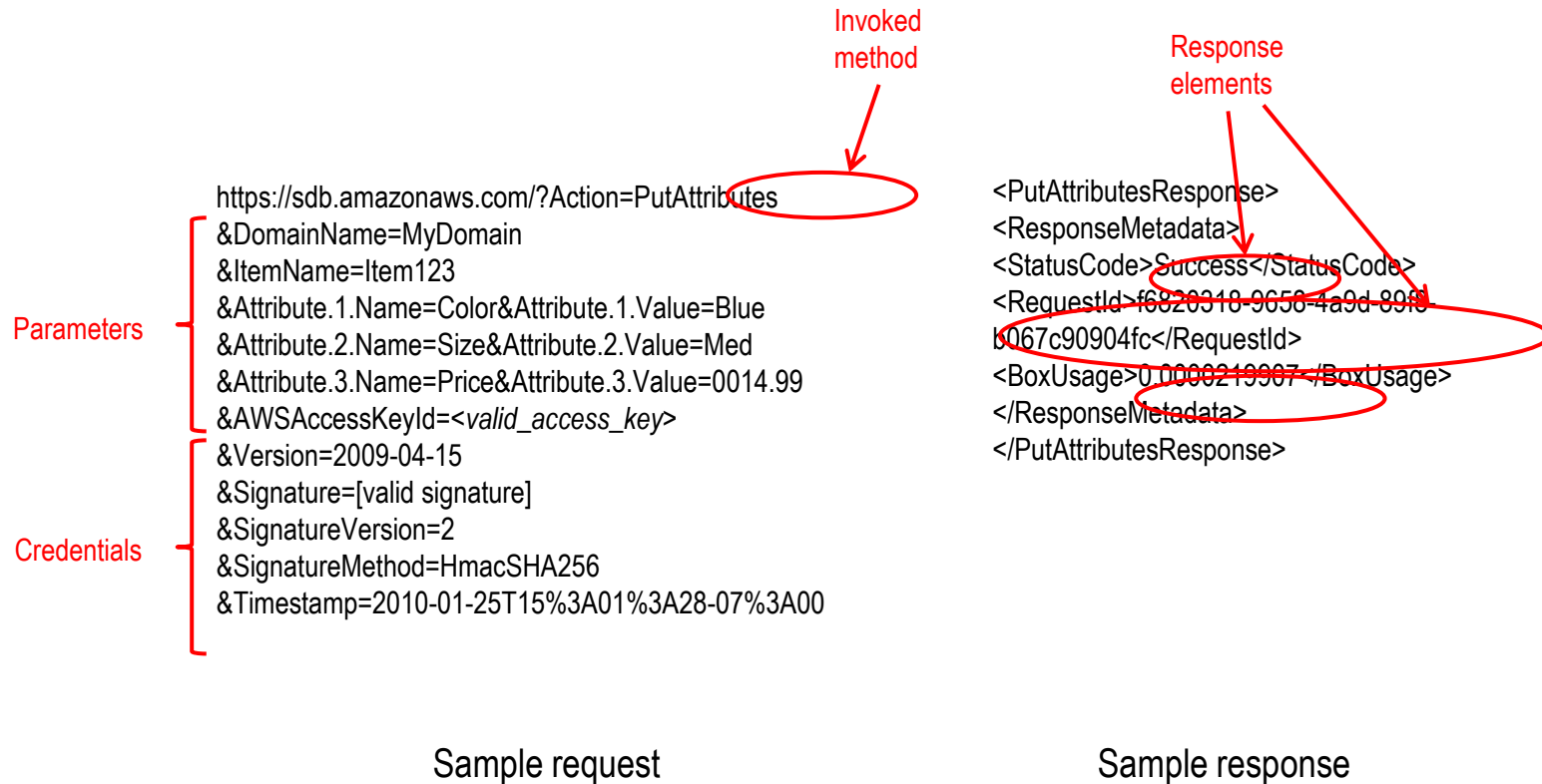
Simple Object Access protocol (SOAP)

- Not as simple as the name suggests, mostly falling out of favor
- XML-based, extensible, general, standardized, but also somewhat heavyweight and verbose

Representational State Transfer (REST)

- Much simpler to develop than SOAP
- Web-specific; uses URLs and GETs to specify parameters where possible; but a lack of standards

Example: REST



Example: SOAP

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
xmlns:SOAP-ENV='http://schemas.xmlsoap.org/soap/envelope/'
xmlns:SOAP-ENC='http://schemas.xmlsoap.org/soap/encoding/'
xmlns:xsi='http://www.w3.org/2001/XMLSchema-instance'
xmlns:xsd='http://www.w3.org/2001/XMLSchema'>
<SOAP-ENV:Body>
<PutAttributesRequest xmlns='http://sdb.amazonaws.com/doc/
2009-04-15'>
<Attribute><Name>a1</Name><Value>2</Value></Attribute>
<Attribute><Name>a2</Name><Value>4</Value></Attribute>
<DomainName>domain1</DomainName>
<ItemName>eID001</ItemName>
<Version>2009-04-15</Version>
</PutAttributesRequest>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

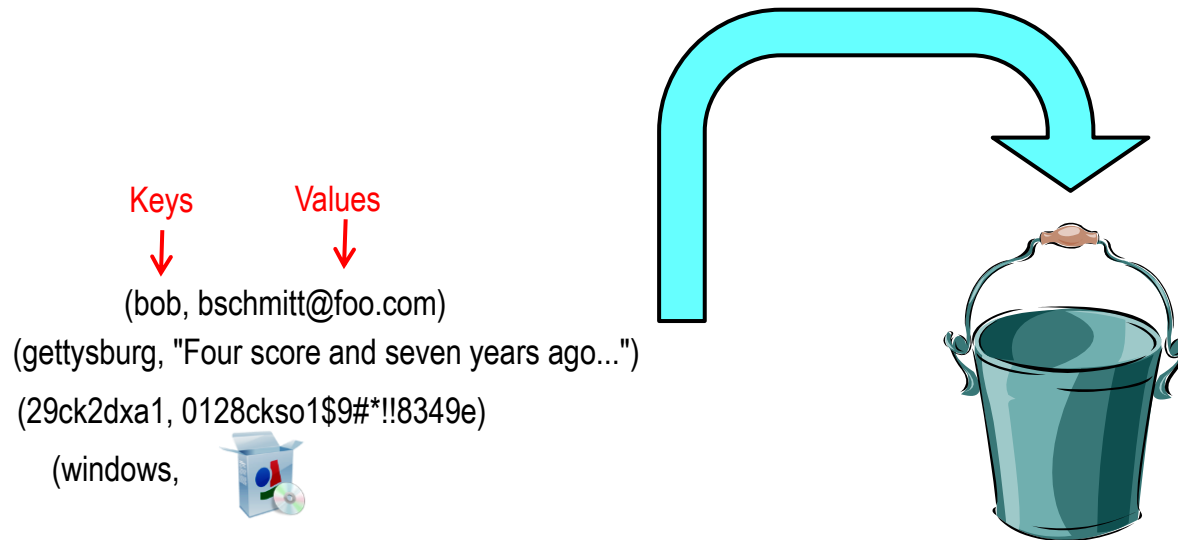
Sample request

```
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Body>
<PutAttributesResponse>
<ResponseMetadata>
<RequestId>4c68e051-fe45-43b2-992a-
a24017ffe7ab</RequestId>
<BoxUsage>0.0000219907</BoxUsage>
</ResponseMetadata>
</PutAttributesResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Sample response

KEY/VALUE STORES AND THE CLOUD

Key-value stores



The **key-value store (KVS)** is a simple **abstraction** for managing persistent state

- Data is organized as (key,value) pairs; sometimes termed a NoSQL database but generally doesn't offer database-like guarantees or properties
- Only three basic operations:

PUT(key, value)

GET(key) → value

Delete(key)

Examples of KVS

Where have you seen this concept before?

Conventional examples outside the cloud:

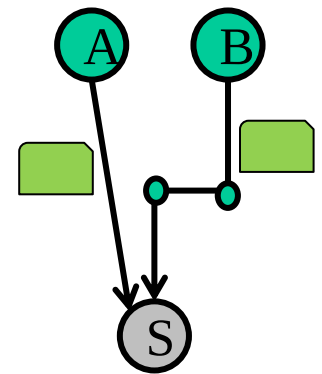
- In-memory **associative arrays** and **hash maps** – limited to a single application, only persistent until program ends
- On-disk indices (BerkeleyDB)
- "Inverted indices" behind search engines
- Database management systems – multiple KVSs++
- Distributed hashtables (e.g., on top of Chord/Pastry)

Supporting an Internet service with a KVS

We'll do this through a central server, e.g., a Web or application server

Two main issues:

1. There may be **multiple concurrent requests** from different clients
These might be GETs, PUTs, DELETES, etc.
2. These requests may come from different parts of the network, with **message propagation delays**
It takes a while for a request to make it to the server!
We'll have to handle requests in the order received (why?)



Managing concurrency in a KVS

What happens if we do multiple GET operations in parallel?

- ... over different keys?
- ... over the same key?

What if we do multiple PUT operations in parallel? or a GET and a PUT?

What is the unit of protection (**concurrency control**) that is necessary here?

Concurrency control

Most systems use **locks** on individual items

- Each requestor asks for the lock
- A **lock manager** processes these requests (typically in FIFO order) as follows:

Lock manager grants the lock to a requestor

Requestor makes modifications

Then releases the lock when it's done



There are several kinds of locks, and several other alternatives

- Example: S/X lock

Limitations of per-key concurrency control

Suppose I want to transfer credits
from my WoW account to my friend's?

- ... while someone else is doing a GET
on my (and her) credit amounts to see if
they want to trade?

This is where one needs a database
management system (DBMS) or transaction
processing manager (app server)

- Allows for “locking” at a higher level, across keys and
possibly even systems

Could you implement higher-level locks within the
KVS? If so, how?



Key-Multi-Value stores

What if I want to have multiple values for the same key in a KVS?

- Example: Multiple images with the same search keyword

Option 1: Make the “value” a collection object like a set

Option 2: Allow the KVS to store multiple values per key

- Often requires a **cursor** that scrolls through the matches

Accessing data with a Cursor

How can we retrieve all the values associated with a particular key?

- Simple case: get back a set
- But what if there are a lot?

A common alternative: Use a **cursor**

- Follows the following programming pattern:

```
cursor = kvs.getFirstMatch(key) ;

while (cursor != null) {
    value = cursor.getValue() ;
    cursor = kvs.getNextMatch(key, cursor) ;
}
```


Recap: Key-value stores

KVS: A simple abstraction for managing persistent data state

- Interface consists only of PUT and GET (+possibly DELETE)
- Some variants allow multiple values per key
- Examples: Distributed hashtables, associative arrays, ...
- Extremely scalable implementations exist

Challenge: Concurrency control

- From the perspective of the KVS, values for different keys are independent
- Difficult to change multiple values atomically
- Some applications may require higher-level locking

Key-Value stores on the Cloud

Many situations need hosting of large data sets

- Examples: Amazon catalog, eBay listings, Facebook pages, ...

Ideal: Abstraction of a 'big disk in the clouds',
which would have:

- Perfect **durability** – nothing would ever disappear in a crash
- 100% **availability** – we could always get to the service
- Zero **latency** from anywhere on earth – no delays!
- Minimal **bandwidth utilization** – we only send across the network what we absolutely need
- **Isolation** under concurrent updates – make sure data stays consistent

The inconveniences of the real world

Why isn't this feasible?

The “cloud” exists over a physical network

- Communication takes time, esp. across the globe
- Bandwidth is limited, both on the backbone and endpoint

The “cloud” has imperfect hardware

- Hard disks crash
- Servers crash
- Software has bugs

Can you map these to the previous desiderata?

Finding the right tradeoff

In practice, we can't have everything

- ... but most applications don't really need 'everything'!

Some observations:

1. **Read-only** (or read-mostly) data is easiest to support

Replicate it everywhere! No concurrency issues!

But only some kinds of data fit this pattern – examples?

2. **Granularity matters**: “Few large-object” tasks generally tolerate longer latencies than “many small-object” tasks

Fewer requests, often more processing at the client

But it's much more expensive to replicate or to update!

3. Maybe it makes sense to develop **separate solutions** for large read-mostly objects vs. small read-write objects!

Different requirements → different technical solutions

Specialized KVS

Cloud KVS are often specialized for a particular tradeoff or usage scenario

Example: Amazon's solutions

- Simple Storage Service (S3):

 - large objects – files, virtual machines, etc.

 - assumes objects change infrequently

 - objects are opaque to the storage system

- SimpleDB (old), DynamoDB (newer replacement):

 - small objects – Java objects, records, etc.

 - generally updated more frequently; greater need for consistency

 - generally multiple attributes or properties, which are exposed to the storage system

Big Objects: Amazon S3

S3 = Simple Storage System

- Think roughly of an Internet file system

Stores large **objects** (=values) that may have **access permissions**

- Used in “cloud backup” services like Jungle Disk, streaming video like Netflix
- Used to distribute software packages
- Used internally by Amazon to store virtual machines

“Up to 99.999999999% durability, 99.99% availability” (“ten nines” and “four nines”)

S3: Key concepts

S3 consists of:

- **objects** – named items stored in S3
- **buckets** of objects – think of these as volumes in a filesystem
- the console includes a notion of **folders**, but these are not intrinsic to S3



Names within a bucket must uniquely identify a single object

- i.e., keys must be unique

S3: Keys and objects

What can we use as keys?

- Keys can be any string

What can we use as objects?

- Objects can be from 1 byte to 5 TB, any format
- Number of objects is 'unlimited'

Where can objects be stored?

- Can be assigned to specific geographic regions (Washington, Virginia, California, Ireland, Singapore, Tokyo, ...)
Why is this important? (name at least four reasons!)

low latency to customer
minimize fault correlation

regulatory/legal requirements
low-storage-cost regions

S3: Different ways to access objects

Objects in S3 can be accessed

- ... via REST or SOAP
- ... via BitTorrent
- ... over the web: <http://s3.amazonaws.com/bucket/key>
- Web Services use HTTP (the Web browser protocol over sockets) and XML to send requests and data
- AWS Console also enables configuration

S3: Access permissions

Permissions are assigned through

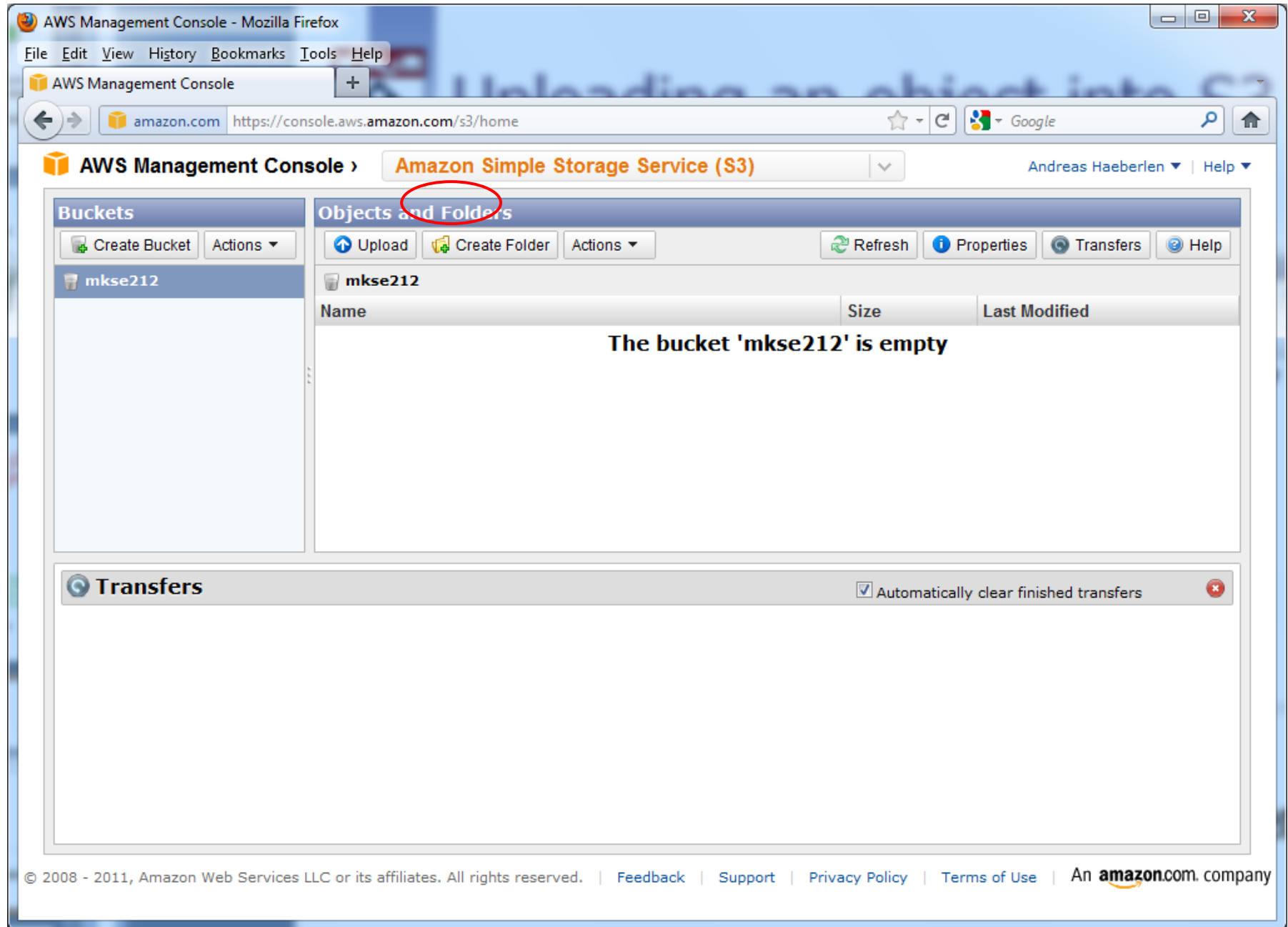
Access Control Lists (ACLs)

- Essentially, a list of users/groups → permissions
- Bucket permissions are inherited by objects unless overridden at the object level

What can you control?

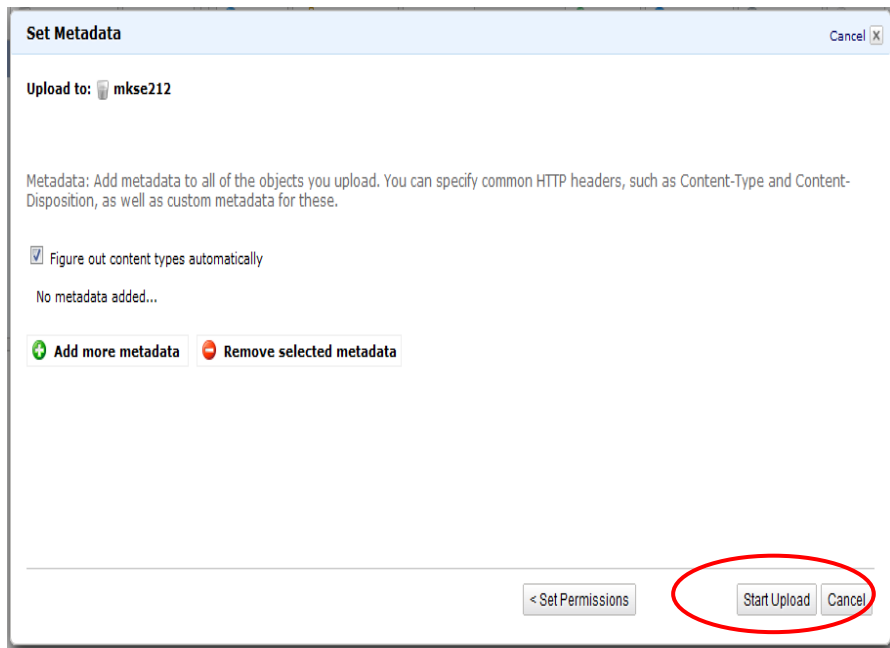
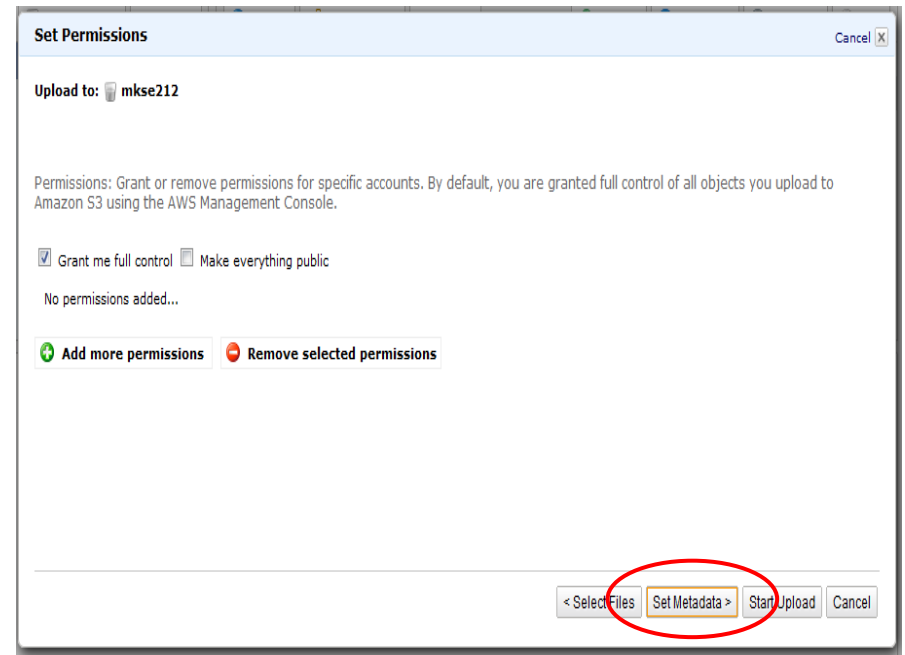
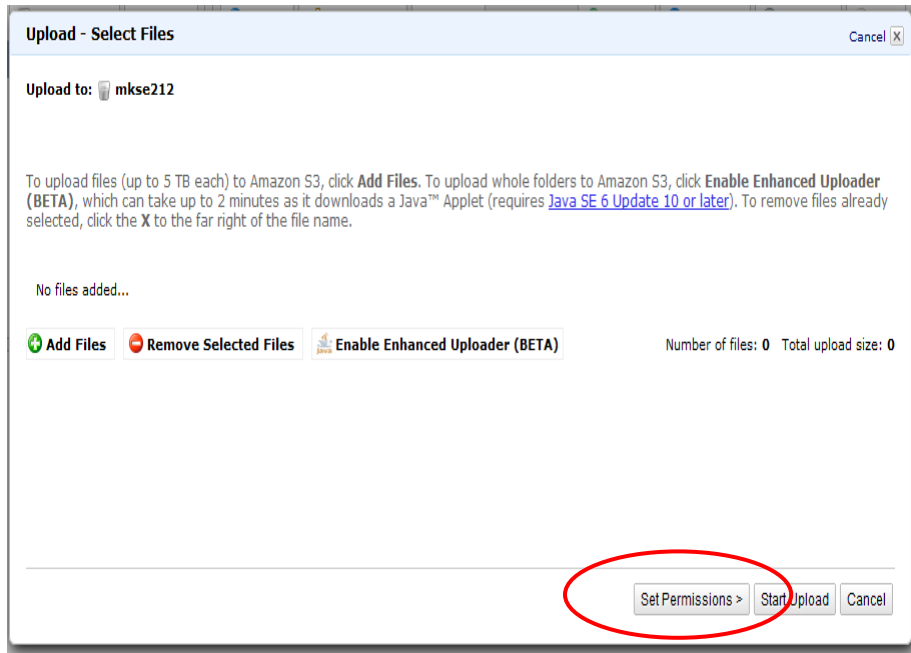
- Can be at the level of buckets or individual objects
- Available rights: Read, write, read ACP, write ACP
- Possible grantees: Everyone, authenticated users, specific users (by AWS account email address)

S3: Uploading an object



Step 1: Hit 'upload' in management console

S3: Uploading an object



Step 2: Select files

Step 3: Set metadata (or accept default)

Step 4: Set permissions (or make public)

S3: Pricing and usage, over a year...

Region:	US Standard ▾		
	Standard Storage	Reduced Redundancy Storage	Glacier Storage
First 1 TB / month	\$0.095 per GB	\$0.076 per GB	\$0.010 per GB
Next 49 TB / month	\$0.080 per GB	\$0.064 per GB	\$0.010 per GB
Next 450 TB / month	\$0.070 per GB	\$0.056 per GB	\$0.010 per GB
Next 500 TB / month	\$0.065 per GB	\$0.052 per GB	\$0.010 per GB
Next 4000 TB / month	\$0.060 per GB	\$0.048 per GB	\$0.010 per GB
Over 5000 TB / month	\$0.055 per GB	\$0.037 per GB	\$0.010 per GB

Request Pricing

Region: US Standard 

Pricing	
PUT, COPY, POST, or LIST Requests	\$0.005 per 1,000 requests
Glacier Archive and Restore Requests	\$0.05 per 1,000 requests
Delete Requests	Free †
GET and all other Requests	\$0.004 per 10,000 requests
Glacier Data Restores	Free ††

† No charge for delete requests of Standard or RRS objects. For objects that are archived to Glacier, there is a pro-rated charge of \$0.03 per gigabyte for objects deleted prior to 90 days. [Learn more.](#)

†† Glacier is designed with the expectation that restores are infrequent and unusual, and data will be stored for extended periods of time. You can restore up to 5% of your average monthly Glacier storage (pro-rated daily) for free each month. If you choose to restore more than this amount of data in a month, you are charged a restore fee starting at \$0.01 per gigabyte. [Learn more.](#)

Data Transfer Pricing

The pricing below is based on data transferred "in" to and "out" of Amazon S3.

Region:	US Standard	▼
Pricing		
Data Transfer IN To Amazon S3		
All data transfer in	\$0.000 per GB	
Data Transfer OUT From Amazon S3 To		
Amazon EC2 in the Northern Virginia Region	\$0.000 per GB	
Another AWS Region or Amazon CloudFront	\$0.020 per GB	
Data Transfer OUT From Amazon S3 To Internet		
First 1 GB / month	\$0.000 per GB	
Up to 10 TB / month	\$0.120 per GB	
Next 40 TB / month	\$0.090 per GB	
Next 100 TB / month	\$0.070 per GB	
Next 350 TB / month	\$0.050 per GB	

	Standard Storage	Reduced Redundancy Storage	Glacier Storage
First 1 TB / month	\$0.0300 per GB	\$0.0240 per GB	\$0.0100 per GB
Next 49 TB / month	\$0.0295 per GB	\$0.0236 per GB	\$0.0100 per GB
Next 450 TB / month	\$0.0290 per GB	\$0.0232 per GB	\$0.0100 per GB
Next 500 TB / month	\$0.0285 per GB	\$0.0228 per GB	\$0.0100 per GB
Next 4000 TB / month	\$0.0280 per GB	\$0.0224 per GB	\$0.0100 per GB
Over 5000 TB / month	\$0.0275 per GB	\$0.0220 per GB	\$0.0100 per GB

Pricing

PUT, COPY, POST, or LIST Requests	\$0.005 per 1,000 requests
Glacier Archive and Restore Requests	\$0.05 per 1,000 requests
Delete Requests	Free †
GET and all other Requests	\$0.004 per 10,000 requests
Glacier Data Restores	Free †

Pricing

Data Transfer IN To Amazon S3

All data transfer in	\$0.000 per GB
----------------------	----------------

Data Transfer OUT From Amazon S3 To

Amazon EC2 in the Northern Virginia Region	\$0.000 per GB
Another AWS Region or Amazon CloudFront	\$0.020 per GB

Data Transfer OUT From Amazon S3 To Internet

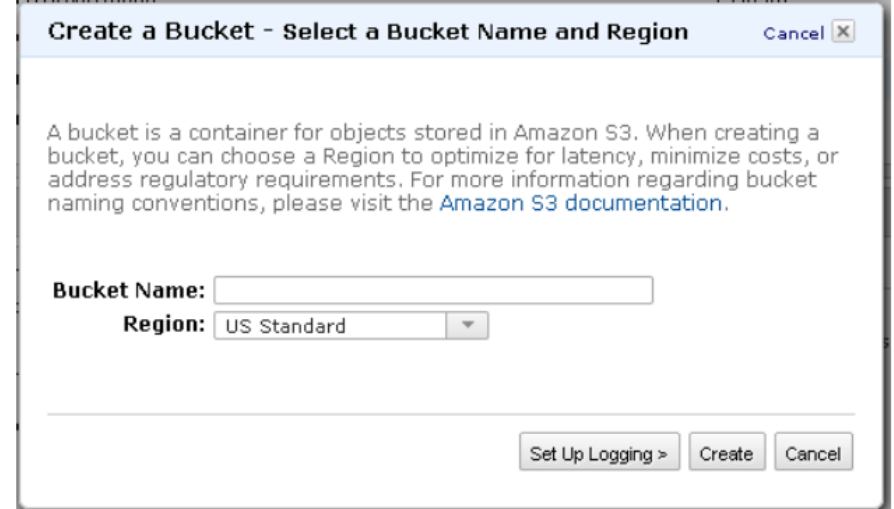
First 1 GB / month	\$0.000 per GB
Up to 10 TB / month	\$0.120 per GB
Next 40 TB / month	\$0.090 per GB
Next 100 TB / month	\$0.070 per GB
Next 350 TB / month	\$0.050 per GB

http://aws.amazon.com/s3/ (9/19/2013)

http://aws.amazon.com/s3/ (9/18/2014)

S3: Bucket operations

- Create bucket
(optionally versioned;
see later)
- Delete bucket
- List all keys in bucket (may not be 100% up to date)
- Modify bucket permissions



The screenshot shows a dialog box titled "Create a Bucket - Select a Bucket Name and Region" with a "Cancel" button in the top right corner. Below the title bar, there is a paragraph of text explaining that a bucket is a container for objects stored in Amazon S3 and that users can choose a Region to optimize for latency, minimize costs, or address regulatory requirements. It also includes a link to "Amazon S3 documentation". Below this text, there are two input fields: "Bucket Name:" followed by a text input box, and "Region:" followed by a dropdown menu currently showing "US Standard". At the bottom right of the dialog, there are three buttons: "Set Up Logging >", "Create", and "Cancel".

Source: Amazon S3 User's Guide

S3: Object operations

- PUT object in bucket
- GET object from bucket
- DELETE object from bucket
- Modify object permissions

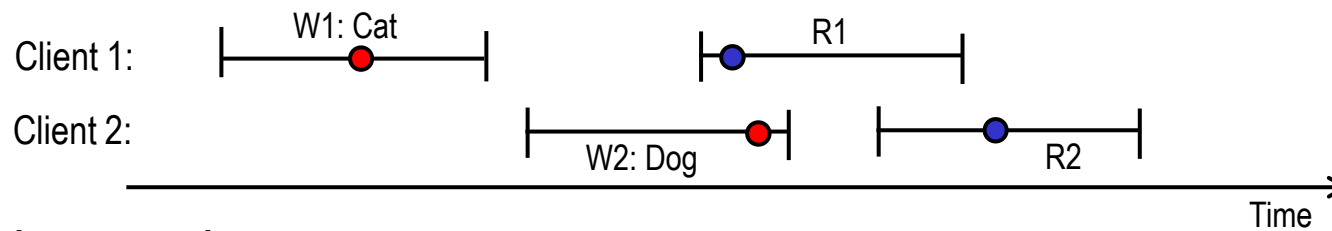
The key issue: How do we manage concurrent updates?

- Will I see objects you delete? the latest version? etc.

S3: Consistency models

Consistency model depends on the region

- US West, EU, Asia Pacific, S. America:
read-after-write consistency for PUTs of new objects and
eventual consistency for overwrite PUTs and DELETES
- S3 buckets in the US Standard Region: **eventual consistency**



Read-after-write consistency:

- Each read or write operation becomes effective at some point between its start time and its completion time
- Reads return the value of the last effective write

S3: Versioning

S3 handles consistency through **versioning** rather than locking

- The idea: every bucket + key maps to a list of versions
[bucket+key] → [object v1] [object v2] [object v3] ...
- Each time we PUT an object, it gets a new version
The last-received PUT overwrites any previous ones!
- When we GET:
An unversioned request likely receives the last version – but this is not guaranteed depending on propagation delays
A request for bucket + key + version uniquely maps to a single object!

Versioning can be enabled for each bucket

- Why would you (not) want versioning?

Recap: Amazon S3

A key-value store for large objects

- Buckets, keys, objects, folders
- Various ways to access objects, e.g., HTTP and BitTorrent

Provides eventual consistency

- +/- a few details that depend on the region

Supports versioning and access control

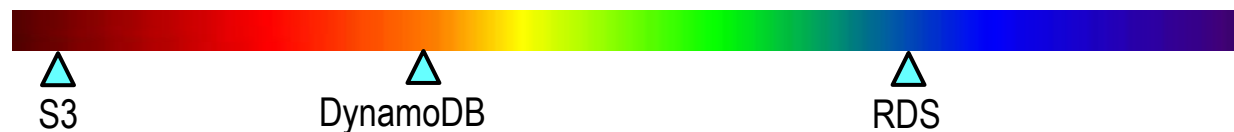
- Access control is based on ACLs

DYNAMODB: RECORD-LIKE KEY- VALUE STORAGE

What is Amazon DynamoDB?

A highly scalable, non-relational data store

- Despite its name, not really a database
- Stronger consistency guarantees than S3
- Highly scalable; built-in replication; automatic indexing
- No 'real' transactions, just a conditional put/delete
- No 'real' relations and joins, just a fairly basic select



DynamoDB: Data model

The diagram shows a table with 9 columns: CustomerID, Date, First name, Last name, Street address, City, State, Zip, and Email. Red arrows point to specific parts of the table with labels: 'Key' points to the CustomerID header; 'Name (hash key)' points to the first '123' in the first two rows; 'Range' points to the Date header; 'Attributes (key-multivalue)' points to the First name, Last name, and Street address headers; and 'Items' points to the first two rows of data.

CustomerID	Date	First name	Last name	Street address	City	State	Zip	Email
123	1/2/3	Bob	Smith	123 Main St	Springfield	MO	65801	
123	2/3/4	Bob	Smith	123 Main St	Kansas City	MO	68041	
456		James	Johnson	456 Front St	Seattle	WA	98104	james@foo.com

Somewhat analogous to a spreadsheet:

- Domains: Entire 'tables'; like buckets
- Items: Names with attribute-multivalue sets
For example, an item could have more than one street address

It is possible to add attributes later

- No pre-defined schema

DynamoDB: Basic operations

List Tables, Get Table Description

Create, Delete Table

GetItem, PutItem, UpdateItem, DeleteItem

- Can do Conditional Writes based on a value

Select (like an SQL query)

DynamoDB: PutItem, UpdateItem, and GetItem

PutItem/UpdateItem has a very simple model:

- Specify the Table, a set of key attributes, and a set of other attributes
- UpdateItem can specify a condition

GetItem

- Specify the Table, set of key attributes
- Can choose whether the read should be strongly consistent or not
- Can also assign a Condition, e.g., that a value matches some equality condition

DynamoDB: Select

A very simple “query” interface based on SQL syntax

- `SELECT output_list FROM domain_name WHERE expression [sort expression] [limit spec]`
- Example: "select * from books where author like 'Tan%' and price <= 55.90 and year is not null order by title desc limit 50"
- Can choose whether or not read should be consistent
- Supports a cursor

Alternatives to SimpleDB

There is a similar service to SimpleDB underneath most major “cloud” companies’ infrastructure

- Google calls theirs BigTable
- Yahoo’s is called PNUTS
- See reading list at the end

All consist of items with a variable set of attribute-value pairs

- More flexible than a relational DBMS table
- But don’t support full-fledged transactions

Alternatives to DynamoDB

There is a similar service to DynamoDB underneath most major “cloud” companies’ infrastructure

- In open source there are platforms like HBase, Cassandra, MongoDB, Accumulo that do similar things
- Google calls theirs BigTable
- Yahoo’s is called PNUTS
- See reading list at the end

All consist of items with a variable set of attribute-value pairs

- More flexible than a relational DBMS table
- But don’t support full-fledged transactions

Recap: Amazon DynamoDB

A scalable, non-relational data store

- Domains, items, keys, values
- Stronger consistency than S3
- No pre-defined schema

Where could we go beyond this?

KVSs present one of the simplest data representations:
key + one or more objects/properties

Some alternatives:

- Relational databases represent data as interlinked tables (in essence, a limited form of a graph)
- Hierarchical storage systems represent data as nested entities
- JSON / Document stores (e.g., MongoDB) support JSON or HTML
- More general graph storage might represent entire graph structures with links

All are implementable over a KVS

- But all allow **higher level requests** (e.g., paths), and might optimize for this

Summary: Cloud Key/Value Stores

Attempt to provide very high **durability**, **availability** in a persistent, geographically distributed storage system

Need to choose **compromises** due to limitations of communications, hardware, software

- Large, seldom-changing objects – **eventual consistency** and versioned model in S3
- Small, more frequently changing objects – lower-latency response, **conditional updates** in DynamoDB

Both are useful in different situations

BEYOND STORAGE: OTHER CLOUD SERVICES

Beyond Storage, What if...

I want to host a Web site?

Or a Web service?

Or an instance of a DBMS that I closely manage?

Amazon (and Azure and Google) give several options, including services they manage (e.g., Amazon RDS) and a bare-bones service *you* manage

- “Infrastructure as a Service”, IaaS
- Amazon Elastic Compute Cloud (EC2), Azure Virtual Machines, Google Compute Engine

Amazon EC2

Logging into AWS Management Console

Launching an instance

Contacting the instance via ssh

Terminating an instance

Have a look at the [AWS Getting Started guide](#)

	vCPU	ECU	Memory (GiB)	Instance Storage (GB)	Linux/UNIX Usage
General Purpose - Current Generation					
t2.micro	1	Variable	1	EBS Only	\$0.013 per Hour
t2.small	1	Variable	2	EBS Only	\$0.026 per Hour
t2.medium	2	Variable	4	EBS Only	\$0.052 per Hour
m3.medium	1	3	3.75	1 x 4 SSD	\$0.070 per Hour
m3.large	2	6.5	7.5	1 x 32 SSD	\$0.140 per Hour
m3.xlarge	4	13	15	2 x 40 SSD	\$0.280 per Hour
m3.2xlarge	8	26	30	2 x 80 SSD	\$0.560 per Hour
Compute Optimized - Current Generation					
c3.large	2	7	3.75	2 x 16 SSD	\$0.105 per Hour
c3.xlarge	4	14	7.5	2 x 40 SSD	\$0.210 per Hour
c3.2xlarge	8	28	15	2 x 80 SSD	\$0.420 per Hour

Oh no - where has my data gone?

EC2 instances do not have persistent storage

- Data survives stops & reboots, but not termination



If you store data on the virtual hard disk of your instance and the instance fails or you terminate it, **your data WILL be lost!**



So where should I put persistent data?

- Elastic Block Store (EBS) - in a few slides
- Ideally, use an AMI with an EBS root (Amazon's default AMI has this property)

Amazon Machine Images

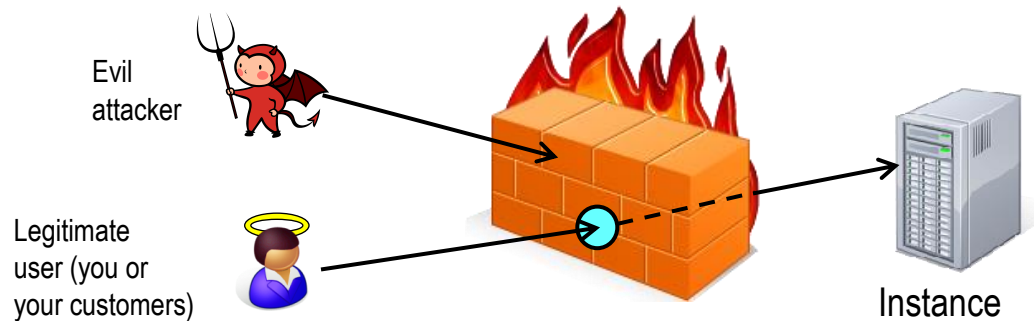
When I launch an instance, what software will be installed on it?

- Software is taken from an [Amazon Machine Image \(AMI\)](#)
- Selected when you launch an instance
- Essentially a file system that contains the operating system, applications, and potentially other data
- Lives in S3

How do I get an AMI?

- Amazon provides several generic ones, e.g., Amazon Linux, Fedora Core, Windows Server, ...
- You can make your own
 - You can even run your own custom kernel (with some restrictions)

Security Groups



Basically, a set of firewall rules

- Can be applied to groups of EC2 instances
- Each rule specifies a protocol, port numbers, etc...
- Only traffic matching one of the rules is allowed through

Sometimes need to explicitly open ports

NETWORK & SECURITY

Security Groups

Elastic IPs

Placement Groups

Load Balancers

Key Pairs

Create a new rule:

Custom TCP rule

Port range:

(e.g., 80 or 49152-65535)

Source:

0.0.0.0/0

(e.g., 192.168.2.0/24, sg-47ad482e, or 1234567890/default)

Add Rule

ICMP	Source	Action
Port (Service)		
ALL	sg-2fc91646 (default)	Delete
TCP	Source	Action
Port (Service)		
0 - 65535	sg-2fc91646 (default)	Delete
22 (SSH)	0.0.0.0/0	Delete
80 (HTTP)	0.0.0.0/0	Delete

Regions and Availability Zones

Where exactly does my instance run?

- No easy way to find out - Amazon does not say

Instances can be assigned to **regions**

- Currently 9 available: US East (Northern Virginia), US West (Northern California), US West (Oregon), EU (Ireland), Asia/Pacific (Singapore), Asia/Pacific (Sydney), Asia/Pacific (Tokyo), South America (Sao Paulo), AWS GovCloud
- Important, e.g., for reducing latency to customers

Instances can be assigned to **availability zones**

- Purpose: Avoid correlated fault
- Several availability zones within each region

Network pricing

AWS does charge for network traffic

- Price depends on source and destination of traffic
- Free within EC2 and other AWS services in **same region** (e.g., S3)

Data Transfer OUT From Amazon EC2 To

Amazon S3, Amazon Glacier, Amazon DynamoDB, Amazon SES, Amazon SQS, or Amazon SimpleDB in the same AWS Region	\$0.00 per GB
---	------------------

Amazon EC2, Amazon RDS, Amazon Redshift or Amazon ElastiCache instances, Amazon Elastic Load Balancing, or Elastic Network Interfaces in the same Availability Zone

Using a private IP address	\$0.00 per GB
----------------------------	------------------

Using a public or Elastic IP address	\$0.01 per GB
--------------------------------------	------------------

Amazon EC2, Amazon RDS, Amazon Redshift or Amazon ElastiCache instances, Amazon Elastic Load Balancing, or Elastic Network Interfaces in another Availability Zone or peered VPC in the same AWS Region	\$0.01 per GB
---	------------------

Another AWS Region or Amazon CloudFront	\$0.02 per GB
---	------------------

Data Transfer OUT From Amazon EC2 To Internet

First 1 GB / month	\$0.00 per GB
--------------------	------------------

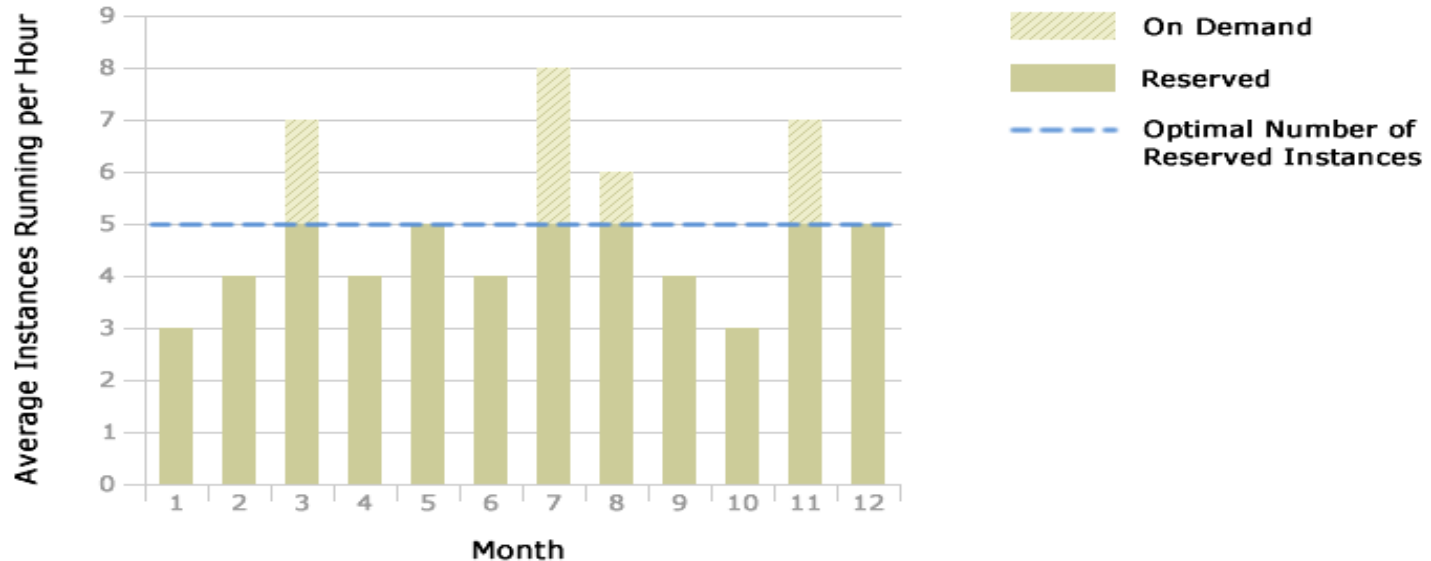
Up to 10 TB / month	\$0.12 per GB
---------------------	------------------

Next 40 TB / month	\$0.09 per GB
--------------------	------------------

Next 100 TB / month	\$0.07 per GB
---------------------	------------------

Next 350 TB / month	\$0.05 per GB
---------------------	------------------

Instance types



So far: **On-demand** instances

Also available: **Reserved** instances

- One-time reservation fee to purchase for 1 or 3 years
- Usage still billed by the hour, but at a considerable discount

Also available: **Spot** instances

- Spot market: Can bid for available capacity
- Instance continues until terminated or price rises above bid

Service Level Agreement

Service Commitment

AWS will use commercially reasonable efforts to make Amazon EC2 and Amazon EBS each available with a Monthly Uptime Percentage (defined below) of at least 99.95%, in each case during any monthly billing cycle (the "Service Commitment"). In the event Amazon EC2 or Amazon EBS does not meet the Service Commitment, you will be eligible to receive a Service Credit as described below.

Definitions

- "Monthly Uptime Percentage" is calculated by subtracting from 100% the percentage of minutes during the month in which Amazon EC2 or Amazon EBS, as applicable, was in the state of "Region Unavailable." Monthly Uptime Percentage measurements exclude downtime resulting directly or indirectly from any Amazon EC2 SLA Exclusion (defined below).
- "Region Unavailable" and "Region Unavailability" mean that more than one Availability Zone in which you are running an instance, within the same Region, is "Unavailable" to you.
- "Unavailable" and "Unavailability" mean:
 - For Amazon EC2, when all of your running instances have no external connectivity.
 - For Amazon EBS, when all of your attached volumes perform zero read write IO, with pending IO in the queue.
- A "Service Credit" is a dollar credit, calculated as set forth below, that we may credit back to an eligible account.

4.38h downtime
per year allowed

Service Commitments and Service Credits

Service Credits are calculated as a percentage of the total charges paid by you (excluding one-time payments such as upfront payments made for Reserved Instances) for either Amazon EC2 or Amazon EBS (whichever was Unavailable, or both if both were Unavailable) in the Region affected for the monthly billing cycle in which the Region Unavailability occurred in accordance with the schedule below.

Monthly Uptime Percentage	Service Credit Percentage
Less than 99.95% but equal to or greater than 99.0%	10%
Less than 99.0%	30%

Recap: EC2

What EC2 is:

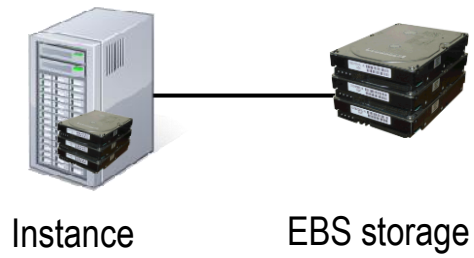
- IaaS service - you can rent virtual machines
- Various types: Very small to very powerful

How to use EC2:

- Ephemeral state - local data is lost when instance terminates
- AMIs - used to initialize an instance (OS, applications, ...)
- Security groups - "firewalls" for your instances
- Regions and availability zones
- On-demand/reserved/spot instances
- Service level agreement (SLA)

VIRTUAL DISKS FOR EC2

Elastic Block Store (EBS)



Persistent storage

- Unlike the local instance store, data stored in EBS is not lost when an instance fails or is terminated

Should I use the instance store or EBS?

- Typically, instance store is used for temporary data

Volumes

EBS storage is allocated in **volumes**

- A volume is a 'virtual disk' (size: 1GB - 1TB)
- Basically, a raw block device
- Can be attached to an instance (but only one at a time)
- A single instance can access multiple volumes

Placed in specific availability zones

- Why is this useful?
- Be sure to place it near instances (otherwise can't attach)

Replicated across multiple servers

- Data is not lost if a single server fails
- Amazon: Annual failure rate is 0.1-0.5% for a 20GB volume

EC2 instances with EBS roots

EC2 instances can have an EBS volume as their root device ("EBS boot")

- Result: Instance data persists independently from the lifetime of the instance
- You can **stop and restart** the instance, similar to suspending and resuming a laptop

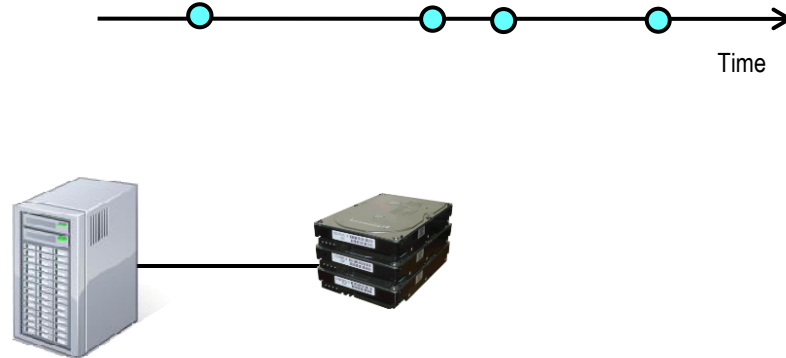
You won't be charged for the instance while it is stopped (only for EBS)

- You can enable **termination protection** for the instance
Blocks attempts to terminate the instance (e.g., by accident) until termination protection is disabled again

Alternative: Use instance store as the root

- You can still store temporary data on it, but it will disappear when you terminate the instance
- You can still create and mount EBS volumes explicitly

Snapshots



You can create a **snapshot** of a volume

- Copy of data in the volume at the time snapshot was made
- Only the first snapshot makes a full copy; subsequent snapshots are incremental

What are snapshots good for?

- Sharing data with others
 - DBpedia snapshot ID is "snap-882a8ae3"
 - Access control list (specific account numbers) or public access
- Instantiate new volumes
- Point-in-time backups

Pricing

You pay for...

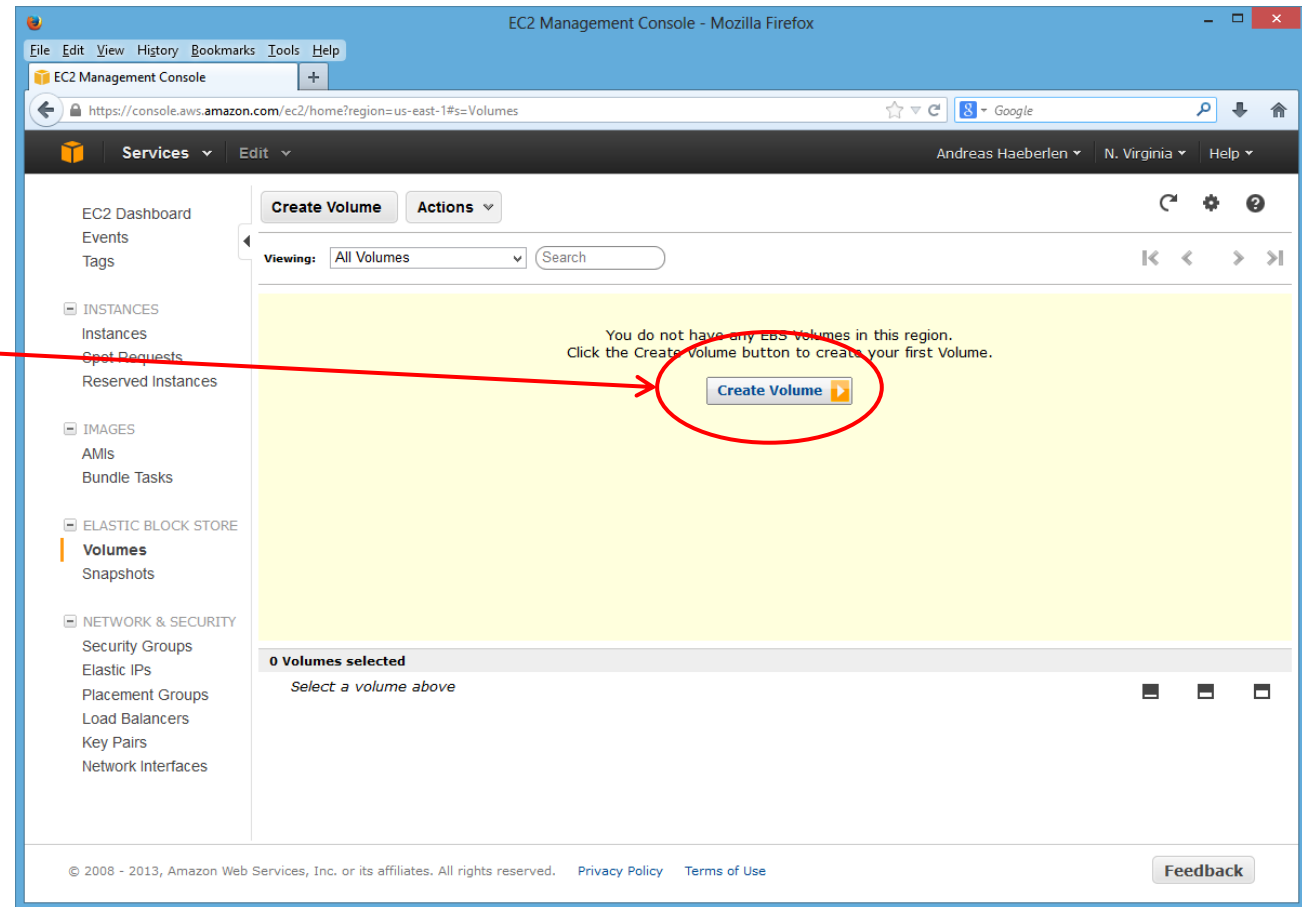
- Storage space: \$0.10 per allocated GB per month
- I/O requests: \$0.10 per million I/O requests
- S3 operations (GET/PUT)

Charge is only for actual storage used

- Empty space does not count

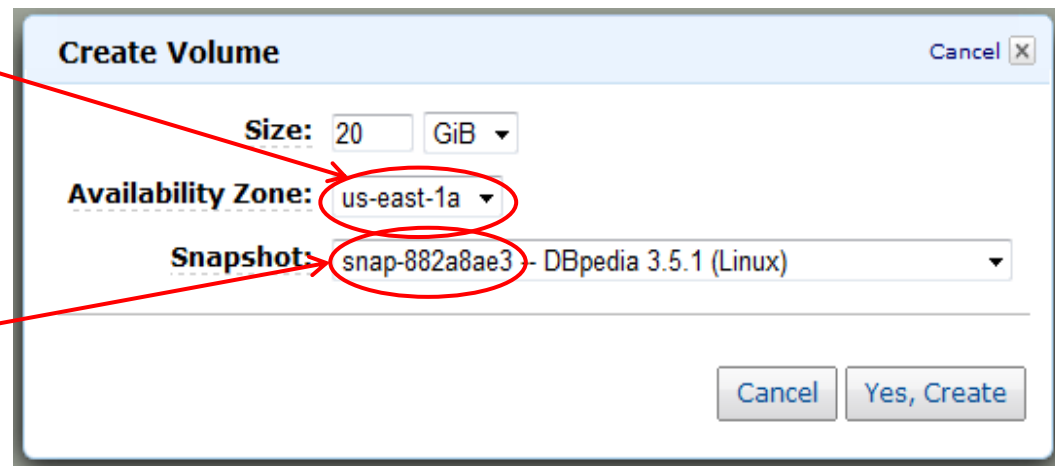
Creating an EBS volume

Create volume



Needs to be in same availability zone as your instance!

DBpedia snapshot ID



Mounting an EBS volume

Step 1: Attach the volume

```
mkse212@vm:~$ ec2-attach-volume -d /dev/sda2 -i i-9bd6eef1
vol-cca68ea5
ATTACHMENT      vol-cca68ea5      i-9bd6eef1      /dev/sda2
attaching
mkse212@vm:~$
```

Step 2: Mount the volume in the instance

```
mkse212@vm:~$ ssh ec2-user@ec2-50-17-64-130.compute-
1.amazonaws.com
```

```
  _ |  _ | _ )  Amazon Linux AMI
 _ |  (  _ | /   Beta
__ | \__ | __ |
```

```
See /usr/share/doc/system-release-2011.02 for latest release
notes. :-)
```

```
[ec2-user@ip-10-196-82-65 ~]$ sudo mount /dev/sda2 /mnt/
```

```
[ec2-user@ip-10-196-82-65 ~]$ ls /mnt/
```

```
dbpedia_3.5.1.owl  dbpedia_3.5.1.owl.bz2  en  other_languages
```

```
[ec2-user@ip-10-196-82-65 ~]$
```


Detaching an EBS volume

Step 1: Unmount the volume in the instance

```
[ec2-user@ip-10-196-82-65 ~]$ sudo umount /mnt/  
[ec2-user@ip-10-196-82-65 ~]$ exit  
mkse212@vm:~$
```

Step 2: Detach the volume

```
mkse212@vm:~$ ec2-detach-volume vol-cca68ea5  
ATTACHMENT          vol-cca68ea5          i-9bd6eef1  
/dev/sda2            detaching  
mkse212@vm:~$
```

Recap: Elastic Block Store (EBS)

What EBS is:

- Basically a virtual hard disk; can be attached to EC2 instances
- Persistent - state survives termination of EC2 instance

How to use EBS:

- Allocate volume - empty or initialized with a snapshot
- Attach it to EC2 instance and mount it there
- Can create snapshots for data sharing, backup

Further reading

- A. Rowstron and P. Druschel: "Storage management and caching in **PAST**, a large-scale, persistent peer-to-peer storage utility" (SOSP'01)
- <http://www.research.microsoft.com/~antr/PAST/past-sosp.pdf>
- F. Chang, J. Dean, S. Ghemawat, W. Hsieh, D. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. Gruber: "**Bigtable**: A Distributed Storage System for Structured Data" (OSDI'06)
- labs.google.com/papers/bigtable-osdi06.pdf
- G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Laksh-man, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels: "**Dynamo**: Amazon's Highly Available Key-Value Store" (SOSP'07)
- <http://dl.acm.org/citation.cfm?id=1294281>
- B. Cooper, R. Ramakrishnan, U. Srivastava, A. Silberstein, P. Bohannon, H. Jacobsen, N. Puz, D. Weaver, and R. Yerneni: "**PNUTS**: Yahoo!'s Hosted Data Serving Platform" (PVLDB'08)
- <http://infolab.stanford.edu/~usriv/papers/pnuts.pdf>
- H. Lim, B. Fan, D. Andersen, and M. Kaminsky: "**SILT**: A Memory-Efficient, High-Performance Key-Value Store" (SOSP'11)
- <http://www.cs.cmu.edu/~dga/papers/silt-sosp2011.pdf>

Big Data

- IDC estimate the size of “digital universe” :
 - 0.18 zettabytes in 2006;
 - 1.8 zettabytes in 2011 (10 times growth);
- A zettabyte is 10^{21} bytes, i.e.,
 - 1,000 exabytes or
 - 1,000,000 petabytes
- Big data is here
 - The ever-growing web!
Machine logs, RFID readers, sensors networks, retail
and enterprise transactions
 - Rich media

New challenges for **storing, managing, and processing** large-scale data

Characteristic by Numbers in 2008

- Max data in memory: 32G
- Max data per computer: 12T
- Data processed by Google a day: 20 PB
- Average job size: 180GB
- Time that takes to read sequentially off a single drive 45 minutes

It's important to work together,
in a simple way.



Questions?

